Module 1

Introduction to Data Analysis in Python

Data Analysis

Burkina Institute of Technology - L3 Computer Science
B. Tanguy KABORE

What we will see

- The DataFrame: the table that holds your data
- Selecting and filtering rows and columns
- Missing values, merging tables, dates
- Statistics, scaling and your first analytics metric

What is data analysis?

Simple definition

Data analysis is the process of inspecting, cleaning, transforming and summarising data to answer a question.

Comparison

Think of a teacher with a pile of exam papers. Reading them one by one is the raw data. Computing the class average, spotting who failed and why, that is the analysis.

Why Python for this?

- One language for loading, cleaning, computing and plotting
- pandas handles tables of millions of rows without a database
- The same code runs on your laptop and on a server
- A huge ecosystem: NumPy, SciPy, Matplotlib, scikit-learn

Excel is fine for 200 rows. Past that, you want code you can rerun.

What is pandas?

Simple definition

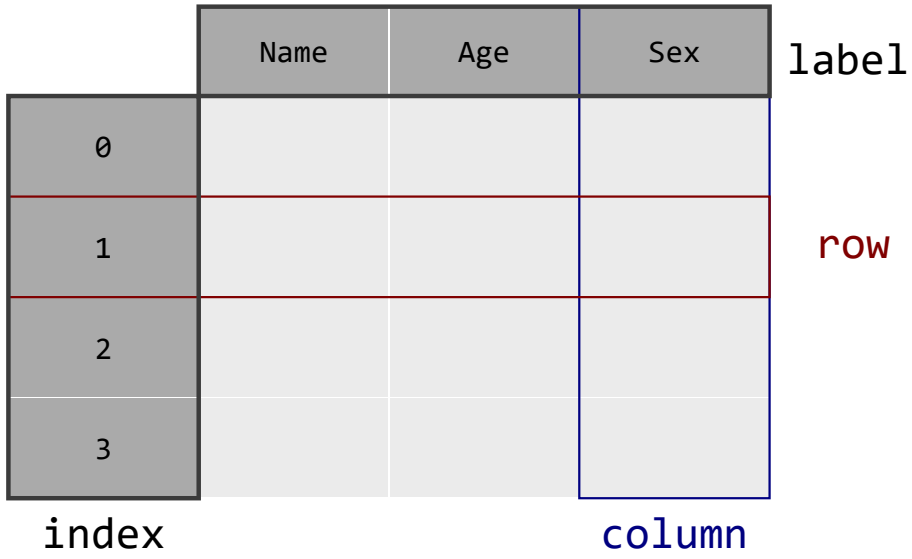
pandas is a Python library built for working with tabular data. Its two main objects are the **Series** (one column) and the **DataFrame** (a whole table).

Convention

Everyone imports it the same way. Do the same so your code is readable by others:

```
import pandas as pd
```

The DataFrame



Source: pandas documentation, "Getting started", https://pandas.pydata.org/docs/ - BSD-3-Clause licence.

Series and DataFrame

Series

- One column of values
- One data type for the whole column
- Has an index (row labels)
- Behaves like a NumPy array with labels

DataFrame

- Several Series sharing the same index
- Each column can have its own type
- Rows and columns both have labels
- The object you will use 90% of the time

Creating a DataFrame

We start with a small table of student grades. Type it with me:

```
import pandas as pd

df = pd.DataFrame({
    "student": ["Awa", "Issa", "Fati", "Moussa", "Rasmata"],
    "dept":    ["CS", "CS", "EE", "CS", "EE"],
    "python":  [15.5, 12.0, None, 9.5, 17.0],
    "stats":   [14.0, 10.5, 13.0, None, 16.0],
})
df
```

Each key becomes a column. `None` will become a missing value, we will come back to it.

What you should see

```
   student dept  python  stats
0      Awa   CS    15.5   14.0
1     Issa   CS    12.0   10.5
2     Fati   EE     NaN   13.0
3   Moussa   CS     9.5    NaN
4  Rasmata   EE    17.0   16.0
```

The numbers 0 to 4 on the left are the **index**. pandas created it for us. `NaN` means "Not a Number": a missing value.

First look at a table

Before any calculation, always run these three:

```
df.head()       # first 5 rows
df.info()       # columns, types, non-null counts
df.describe()   # statistics of numeric columns
```

This takes ten seconds and saves you an hour later. You learn how many rows you have, which columns are numeric and where values are missing.

Reading df.info()

```
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   student  5 non-null      object
 1   dept     5 non-null      object
 2   python   4 non-null      float64
 3   stats    4 non-null      float64
```

`object` means text. `float64` means decimal numbers. `4 non-null` out of 5 rows tells you one value is missing.

Reading df.describe()

```
          python      stats
count   4.000000   4.000000
mean   13.500000  13.375000
std     3.391165   2.286737
min     9.500000  10.500000
50%    13.750000  13.500000
max    17.000000  16.000000
```

`count` ignores missing values. `50%` is the median. If `mean` and `50%` are far apart, the distribution is skewed: look for outliers.

Selecting one column

```
df["python"]
```

```
0    15.5
1    12.0
2     NaN
3     9.5
4    17.0
Name: python, dtype: float64
```

The result is a **Series**. You can also write `df.python`, but the bracket form works with any column name, including names with spaces.

Selecting several columns



```
df[["student", "python"]]      # note the double brackets
```

Source: pandas documentation, "How do I select a subset of a DataFrame?" - BSD-3-Clause licence.

Common error: wrong column name

```
df["Python"]
```

```
KeyError: 'Python'
```

Cause: column names are case sensitive. The column is `python`, not `Python`.

Fix: check the exact names with `df.columns`. When a file comes from someone else, I run `df.columns = df.columns.str.strip().str.lower()` right after loading it.

Two ways to index: loc and iloc

`.loc` - by label

```
df.loc[1, "student"]
# 'Issa'
df.loc[0:2, ["student", "dept"]]
```

Uses the index labels and column names. The end of a slice is included.

`.iloc` - by position

```
df.iloc[0:2, 0:2]
```

```
  student dept
0     Awa   CS
1    Issa   CS
```

Pure integer positions, like a Python list. End excluded.

Filtering rows with a condition

The question: who scored more than 12 in Python?

```
df[df["python"] > 12]
```

```
   student dept  python  stats
0      Awa   CS    15.5   14.0
4  Rasmata   EE    17.0   16.0
```

`df["python"] > 12` produces a Series of True/False. Putting it inside the brackets keeps only the True rows.

Combining conditions

```
df[(df["dept"] == "CS") & (df["python"] >= 10)]
```

Three rules that cause most of the bugs in this room:

1. Use `&` for AND, `|` for OR, `~` for NOT. Not `and`, `or`, `not`.
2. Put parentheses around each condition.
3. Compare with `==`, not `=`.

Error you will meet

```
df[df["dept"] == "CS" & df["python"] >= 10]
```

```
TypeError: Cannot perform 'rand_' with a dtyped [float64] array
and scalar of type [bool]
```

Cause: without parentheses, Python evaluates `"CS" & df["python"]` first, which makes no sense.

Fix: `(df["dept"] == "CS") & (df["python"] >= 10)`.

If you write `and` instead of `&`, you get another message: `ValueError: The truth value of a Series is ambiguous`. Same fix.

Changing the index

```
df2 = df.set_index("student")
df2.loc["Fati"]
```

```
dept         EE
python      NaN
stats      13.0
Name: Fati, dtype: object
```

Now the student name is the row label. Use `reset_index()` to go back to the numeric index.

Exercise 1 - Selection (15 minutes)

Do it now

1. Create the grades DataFrame from the previous slides
2. Display only students from the EE department
3. Display the `student` and `stats` columns for rows 1 to 3 with `.iloc`
4. Display the students whose Python grade is below the class mean

Write each result in a separate cell. We correct together in 15 minutes.

Missing data

Why it matters

A single missing value can change an average or make a function crash. You must know where the holes are before you compute anything.

Where it comes from

A student absent at the exam, a sensor that stopped, a form field left empty, two files merged with different keys.

Finding missing values

```
df.isna().sum()
```

```
student    0
dept       0
python     1
stats      1
dtype: int64
```

`isna()` gives True where a value is missing. `sum()` counts the True values per column. Run this on every new dataset.

Strategy 1: drop the rows

```
df.dropna()
```

```
   student dept  python  stats
0      Awa   CS    15.5   14.0
1     Issa   CS    12.0   10.5
4  Rasmata   EE    17.0   16.0
```

Simple, but we lost 2 students out of 5. On a large dataset with many columns, `dropna()` can delete most of your data. Check the shape before and after.

Strategy 2: fill the holes

```
df.fillna({"python": df["python"].mean(),
           "stats":  df["stats"].mean()})
```

```
   student dept  python   stats
2     Fati   EE    13.5  13.000
3   Moussa   CS     9.5  13.375
```

Replacing by the mean keeps the row but invents a value. Say it clearly in your report. For categories, use the mode; for time series, `ffill()` is often better.

Which strategy?

Situation	Reasonable choice
Few missing rows, large dataset	`dropna()`
Numeric column, roughly symmetric	fill with mean
Numeric column with outliers	fill with median
Category	fill with mode or "unknown"
The absence itself is information	keep it, add a flag column

There is no universal answer. Write down what you did and why.

Merging DataFrames

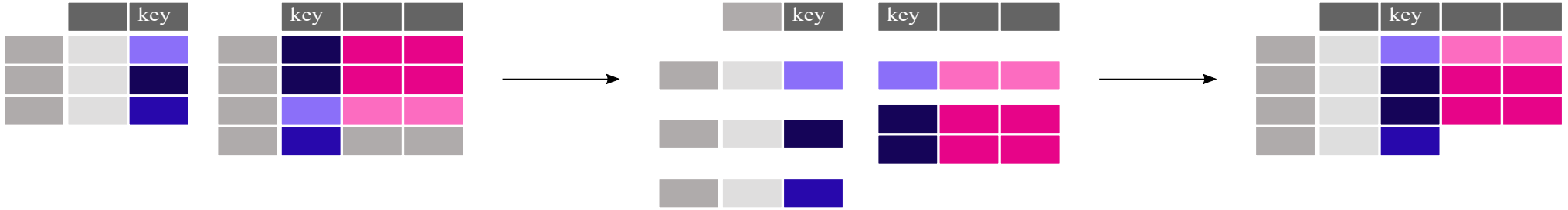
The question

Grades are in one table, department names in another. We want both in a single table.

```
depts = pd.DataFrame({
    "dept": ["CS", "EE", "ME"],
    "dept_name": ["Computer Science", "Electrical Eng.", "Mechanical Eng."],
})
```

This is exactly a SQL JOIN. The shared column `dept` is the key.

Left merge



Source: pandas documentation, "How to combine data from multiple tables?" - BSD-3-Clause licence.

Merging in practice

```
pd.merge(df, depts, on="dept", how="left")
```

```
   student dept  python  stats         dept_name
0      Awa   CS    15.5   14.0  Computer Science
2     Fati   EE     NaN   13.0   Electrical Eng.
4  Rasmata   EE    17.0   16.0   Electrical Eng.
```

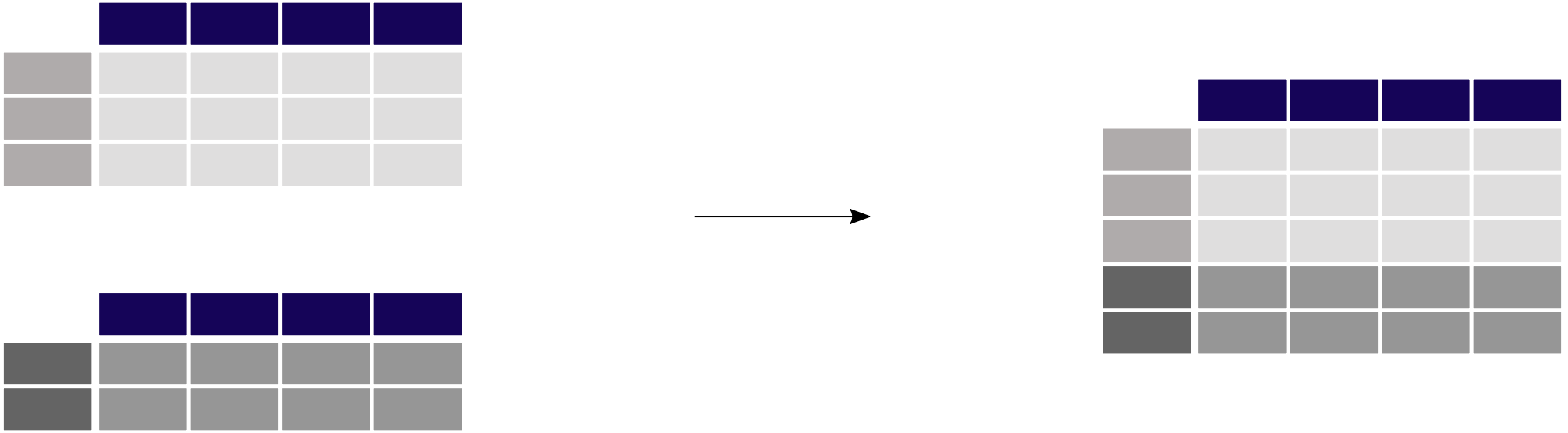
`how="left"` keeps every student. `ME` has no student, so it does not appear. With `how="inner"`, only keys present on both sides survive.

The four kinds of merge

`how=`	Rows kept
`inner`	Keys present in both tables (default)
`left`	All rows of the left table
`right`	All rows of the right table
`outer`	All keys from both tables, NaN where missing

Always check `.shape` before and after a merge. Rows that multiply mean your key is not unique on one side.

Stacking tables: concat



```
pd.concat([df_group_a, df_group_b], ignore_index=True)
```

Use `concat` when two tables have the same columns and you want to put one under the other, for example grades of two groups.

Source: pandas documentation - BSD-3-Clause licence.

Working with dates

Dates arrive as text. Text cannot be sorted by month or subtracted. First step, always:

```
sales = pd.DataFrame({
    "date":   ["2026-01-15", "2026-01-28", "2026-02-03",
               "2026-02-20", "2026-03-07"],
    "amount": [12000, 8500, 15000, 9000, 11000],
})
sales["date"] = pd.to_datetime(sales["date"])
sales.dtypes
```

```
date      datetime64[ns]
amount             int64
```

Extracting parts of a date

```
sales["month"] = sales["date"].dt.month
sales
```

```
        date  amount  month
0 2026-01-15   12000      1
1 2026-01-28    8500      1
2 2026-02-03   15000      2
```

The `.dt` accessor gives `.year`, `.month`, `.day`, `.dayofweek`, `.quarter`. Subtracting two dates gives a duration: `(sales["date"].max() - sales["date"].min()).days` returns `51`.

Grouping by period: resample

```
sales.set_index("date").resample("ME")["amount"].sum()
```

```
date
2026-01-31    20500
2026-02-28    24000
2026-03-31    11000
```

`"ME"` means month end (pandas 2.2 and later; older versions used `"M"`). Other codes: `"W"` week, `"QE"` quarter, `"YE"` year. `resample` needs a datetime index.

Date formats from real files

Files from Burkina Faso and France often use day/month/year. Tell pandas:

```
pd.to_datetime(s, format="%d/%m/%Y")        # 15/01/2026
pd.to_datetime(s, dayfirst=True)           # ambiguous dates
```

Without this, `01/02/2026` is read as January 2nd. Nothing crashes, the result is silently wrong. This is the worst kind of bug: always check a few dates by hand.

Exercise 2 - Cleaning and merging (20 minutes)

Do it now

1. Count the missing values in the grades table and fill them with the column median
2. Merge with the `depts` table and keep all departments (`how="outer"`)
3. Create a `sales` table with 6 dates in `dd/mm/yyyy` format and convert them
4. Compute the total amount per month

Keep the notebook; we reuse it after the break.

Statistics on a DataFrame

```
df["python"].mean()      # 13.5
df["python"].median()    # 13.75
df["python"].std()       # 3.39
df["python"].min(), df["python"].max()
```

These methods skip `NaN` by default. On the whole table, `df.mean(numeric_only=True)` computes one value per numeric column.

Error: statistics on text

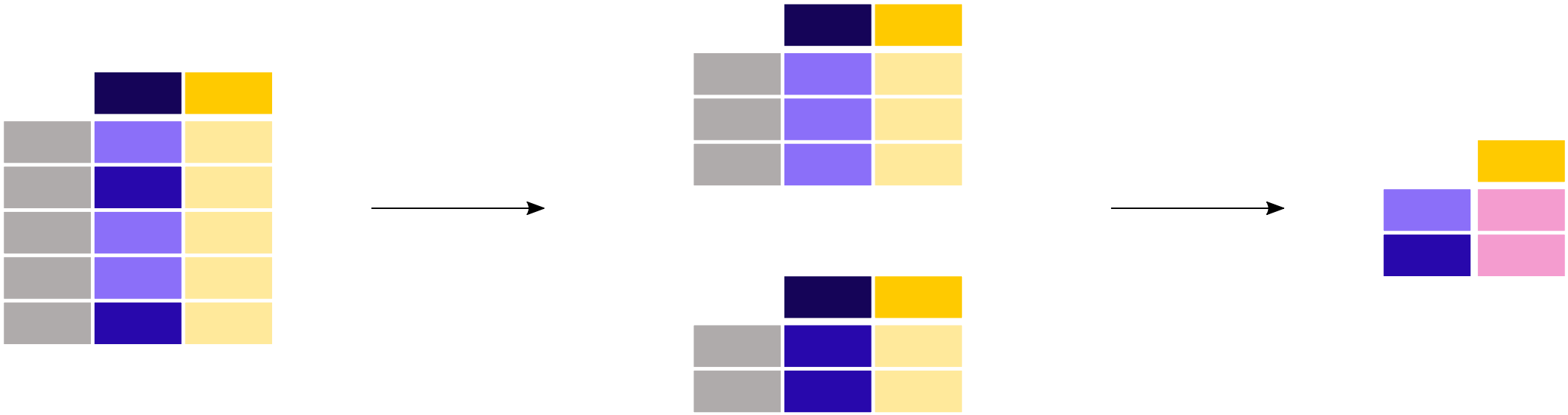
```
df["student"].mean()
```

```
TypeError: Could not convert string 'AwaIssaFatiMoussaRasmata' to numeric
```

Cause: pandas tried to add the names together. This also happens when a numeric column was read as text because of a stray character like `"15,5"` or `"N/A"`.

Fix: check `df.dtypes`, then `pd.to_numeric(df["col"], errors="coerce")` to convert and turn bad values into NaN.

Group by



Source: pandas documentation, "How to calculate summary statistics?" - BSD-3-Clause licence.

Group by in practice

```
df.groupby("dept")["python"].mean()
```

```
dept
CS    12.333333
EE    17.000000
```

Several statistics at once:

```
df.groupby("dept")[["python", "stats"]].agg(["mean", "count"])
```

`count` next to `mean` tells you how many students are behind each average. An average of 17 computed on one student means nothing.

Measurement scales

Before computing, ask: what kind of values is this column?

Scale	Example	Allowed
Nominal	department, city	count, mode
Ordinal	grade letter A/B/C, satisfaction 1-5	median, rank
Interval	temperature in Celsius	mean, difference
Ratio	amount in FCFA, age	everything, incl. ratios

Computing the mean of a department code is a classic beginner error. pandas will not stop you.

Why standardise?

Suppose we want a score mixing an age (18 to 60) and a salary (100 000 to 2 000 000 FCFA). If we add them, the salary crushes the age.

Two columns on different scales cannot be compared or combined directly. We bring them to a common scale first.

Min-max scaling

Brings values between 0 and 1:

```
s = df["python"]
(s - s.min()) / (s.max() - s.min())
```

```
0    0.80
1    0.33
3    0.00
4    1.00
```

Easy to read. Sensitive to outliers: one extreme value squeezes everyone else near 0.

Z-score standardisation

Centres on 0, with a standard deviation of 1:

```
(s - s.mean()) / s.std()
```

```
0    0.59
1   -0.44
3   -1.18
4    1.03
```

A z-score of -1.18 means "1.18 standard deviations below the mean". Values beyond 3 or -3 deserve a look: likely outliers or input errors.

Creating an analytics metric

A metric is a number you compute to answer a business or academic question. Example: did the student pass?

```
df["average"] = df[["python", "stats"]].mean(axis=1)
df["passed"]  = df["average"] >= 10
df["passed"].mean()        # 0.8  -> 80 % pass rate
```

`axis=1` means "across columns, one value per row". The mean of a boolean column is a proportion. This trick is used everywhere.

Metric result

```
   student dept  python  stats  average  passed
0      Awa   CS    15.5   14.0    14.75    True
1     Issa   CS    12.0   10.5    11.25    True
2     Fati   EE     NaN   13.0    13.00    True
3   Moussa   CS     9.5    NaN     9.50   False
4  Rasmata   EE    17.0   16.0    16.50    True
```

Notice: Fati's average is based on one grade only, because `mean` skipped the NaN. Is that fair? That is a decision for you, not for pandas.

Warning: modifying a filtered table

```
sub = df[df["dept"] == "CS"]
sub["python"] = 0
```

```
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

Cause: `sub` may be a copy or a view of `df`; pandas cannot promise which. Your change may not reach `df`.

Fix: `df.loc[df["dept"] == "CS", "python"] = 0` to modify `df`, or `sub = df[...].copy()` to work on an independent table.

Practical work 1 - Student results (1h30)

Goal

Produce a clean results table and a short summary for the head of department.

Material

A CSV of 60 fictitious students, 3 subjects, provided in class. Some grades missing, dates of birth in `dd/mm/yyyy`, a separate CSV of departments.

Work in pairs. Deliverable: one notebook that runs from top to bottom.

Practical work 1 - Steps

1. Load both files, run `info()` and `describe()`, note the problems you see
2. Fix column names, convert dates, compute the age of each student
3. Count missing grades per subject and choose a strategy; justify it in a markdown cell
4. Merge with the departments table and check the shape

Practical work 1 - Analysis

1. Compute the weighted average per student (weights given in class)
2. Add a `passed` column and compute the pass rate per department
3. Standardise the grades of each subject with a z-score and find students more than 2 standard deviations below the mean
4. Write 5 lines of conclusions for the head of department

Practical work 1 - Checks before you submit

- `df.shape` after the merge equals the number of students
- No `SettingWithCopyWarning` left in the output
- Every missing value you filled is documented
- The notebook runs with "Restart kernel and run all"

Summary

What you know now

- A DataFrame is a labelled table; `info()` and `describe()` come first
- `[]`, `.loc` and `.iloc` select; boolean masks filter
- `isna()`, `dropna()`, `fillna()`, `merge()`, `to_datetime()` clean and combine
- `groupby()`, scaling and derived columns turn data into metrics

Next module

Module 2: Graphical Data Representation with Matplotlib

References

- pandas documentation, User Guide: https://pandas.pydata.org/docs/user_guide/
- pandas, "10 minutes to pandas": https://pandas.pydata.org/docs/user_guide/10min.html
- A. Boschetti, L. Massaron, *Python Data Science Essentials*, 3rd ed., Packt, 2018, chapter 2
- Examples tested with pandas 2.2.3

Questions?

Give me one real situation where filling missing values with the mean would be a bad idea.